

Predicting Accident Severity for Risk Assessment in Car Rentals

Course: BBM110L Candidate number: 857784

Introduction

Business Problem

The client is a car rental company which is facing financial and reputational damage due to road accidents. These accidents are leading to direct costs like vehicle repair, and higher insurance premiums. They are also causing indirect costs like legal liability and loss of brand trust. Right now, the company doesn't have a data-driven procedure to assess the how much risk there is in renting cars. The ability to predict how severe an accident can be before the car is rented would allow the company to make informed decisions of whether or not to allow a transaction, and if they should allow it then what precautions to take.

Practical Significance

I am trying to train machine learning models that can predict the severity of an accident using data about the driver, the car, and driving conditions. By accurately predicting severity, the company can do due diligence on the driver and the scenario so they that can take action to reduce liability. These can include adjusting insurance pricing so that premiums can be issued according to risk. Another operation is fleet maintenance which can let the company identify if a vehicle's age or type correlate with severe accidents, this can allow them to check which vehicles need to be removed or kept. In extreme cases, the company can also do proactive risk management by denying rentals for high-risk scenarios like for inexperienced drivers in high powered vehicles during bad weather. Or they can demand additional insurance coverage.

Exploratory Data Analysis (EDA)

To understand the underlying data structure, I performed ED on the dataset.

Data Quality and NaN Identification

During the inspection, I found that there were placeholder values in place of instead of normal NaN (null) values. The dataset used indicators like -1, Data missing or out of range, and Not known. The Age_of_Vehicle and Propulsion_Code columns had many instances of -1. To fix this I created a function to scan all the columns for these placeholders. These were flagged for conversion to NaN during the preprocessing stage to save the model from interpreting -1 as a valid numerical age and getting an incorrect answer.

Target Variable Distribution

The target variable, Accident_Severity, is imbalanced. The majority of records belong to Class 3, while Class 1 represents a minority. This implies that a standard model might get high accuracy by simply predicting "Slight" every time thereby getting high recall, but it would fail to identify fatal accidents leading to low precision. This led to using class_weight='balanced' and stratified sampling in the model development phase.

Feature Correlation

then I did correlation analysis to find linear relationships between features and severity. The correlation matrix showed that linear correlations between most of the features and the target were weak. This implies that the relationship between driving conditions and accident severity is non-linear, so tree-based models like Random Forest and Gradient Boosting would be better for predicting than linear models like Logistic Regression.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

# Sklearn imports
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import OneHotEncoder, RobustScaler,
LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay, f1_score, accuracy_score

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
HistGradientBoostingClassifier

# Global Configuration
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)
plt.style.use('ggplot')

# Constants
RANDOM_STATE = 42
MISSING_VALUES = [-1, -1.0, 'Data missing or out of range', 'Not
known', 'Undefined', 'Unclassified']

%%javascript
var nb = IPython.notebook;
```

```

var kernel = IPython.notebook.kernel;
var command = "NOTEBOOK_FULL_PATH = '" + nb.notebook_path + "'";
kernel.execute(command);

import io
from nbformat import read, NO_CONVERT

with io.open(NOTEBOOK_FULL_PATH.split("/")[-1], 'r', encoding='utf-8')
as f:
    nb = read(f, NO_CONVERT)

word_count = 0
for cell in nb.cells:
    if cell.cell_type == "markdown":
        word_count += len(cell['source'].replace('#',
        '').lstrip().split(' '))
print(f"Word count: {word_count}")

<IPython.core.display.Javascript object>

# Load Data
try:
    accidents = pd.read_csv('1_accidents.csv')
    vehicles = pd.read_csv('1_vehicles.csv')
    df = pd.merge(accidents, vehicles, on='Accident_Index',
how='inner')
    print(f"Data Loaded Successfully. Shape: {df.shape}")
except FileNotFoundError:
    print("Error: CSV files not found. Please upload '1_accidents.csv'
and '1_vehicles.csv'")

# Define Features (X) and Target (y)
X = df.drop(columns=['Accident_Severity'])
y = df['Accident_Severity']

# Split Data (80% Train, 20% Test)
# Reduced test size to 0.2 to give the model more 'Fatal' cases to
learn from
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print(f"Training Shape: {X_train.shape}")
print(f"Test Shape: {X_test.shape}")

Data Loaded Successfully. Shape: (7821, 20)
Training Shape: (6256, 19)
Test Shape: (1565, 19)

def clean_for_eda(df_raw, target):
    """Helper to clean data just for visualization purposes."""

```

```

df = df_raw.copy()
df['Accident_Severity'] = target

# Handle Placeholders
df = df.replace(MISSING_VALUES, np.nan)

# Engineer Hour
if 'Datetime' in df.columns:
    df['Hour'] = pd.to_datetime(df['Datetime']).dt.hour

return df

def plot_proportional_numeric(df, feature, target, bins=None,
labels=None, xlabel=None):
    """
    Plots a proportional stacked bar chart for a numerical feature
    against the target.
    If 'bins' is provided, it groups the data first.
    """
    df_temp = df.copy()

    # Drop NaNs in the feature to avoid plotting errors
    df_temp = df_temp.dropna(subset=[feature])

    # Binning Strategy
    if bins is not None:
        col_name = f'{feature}_group'
        df_temp[col_name] = pd.cut(df_temp[feature], bins=bins,
labels=labels, include_lowest=True)
    else:
        col_name = feature # Use raw values (e.g. Speed_limit)

    # Groupby and Pivot
    counts = df_temp.groupby([col_name,
target]).size().unstack(fill_value=0)

    # Calculate Percentage (Row-wise)
    pcts = counts.div(counts.sum(axis=1), axis=0) * 100

    # Plot
    ax = pcts.plot(kind='bar', stacked=True, colormap='RdYlGn_r',
figsize=(12, 6))

    plt.title(f'Proportional Accident Severity by {xlabel if xlabel
else feature}')
    plt.ylabel('Percentage %')
    plt.xlabel(xlabel if xlabel else feature)
    plt.legend(title='Severity', labels=['Fatal', 'Serious',
'Slight'], bbox_to_anchor=(1.02, 1), loc='upper left')

```

```

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# --- EXECUTE PLOTS ---

# 1. Speed Limit (Treat as discrete, no binning needed)
speed_bins = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
speed_labels = ['10', '20', '30', '40', '50', '60', '70', '80', '90+']
plot_proportional_numeric(X_train.join(y_train), 'Speed_limit',
                           'Accident_Severity',
                           bins=speed_bins, labels=speed_labels,
                           xlabel='speed Group')

# 2. Age of Driver (Needs Binning)
# Bins: <17, 17-24 (Young), 25-64 (Adult), 65+ (Senior)
age_bins = [0, 17, 24, 64, 100]
age_labels = ['<17', '17-24', '25-64', '65+']
plot_proportional_numeric(X_train.join(y_train), 'Age_of_Driver',
                           'Accident_Severity',
                           bins=age_bins, labels=age_labels,
                           xlabel='Driver Age Group')

# 3. Age of Vehicle (Needs Binning)
# Bins: New(0-3), Mid(4-10), Old(10+)
veh_bins = [0, 3, 10, 50]
veh_labels = ['0-3 Years', '4-10 Years', '10+ Years']
plot_proportional_numeric(X_train.join(y_train), 'Age_of_Vehicle',
                           'Accident_Severity',
                           bins=veh_bins, labels=veh_labels,
                           xlabel='Vehicle Age Group')

# 4. Engine Capacity (Needs Binning)
# Bins: Small(<1.5L), Medium(1.5-2.0L), Large(2.0L+)
cc_bins = [0, 1500, 2000, 10000]
cc_labels = ['Small (<1.5L)', 'Medium (1.5-2.0L)', 'Large (>2.0L)']
plot_proportional_numeric(X_train.join(y_train),
                           'Engine_Capacity_(CC)', 'Accident_Severity',
                           bins=cc_bins, labels=cc_labels,
                           xlabel='Engine Size')

def plot_eda(X_data, y_data):
    df_viz = clean_for_eda(X_data, y_data)

    # 1. Target Distribution
    plt.figure(figsize=(8, 5))
    sns.countplot(x='Accident_Severity', data=df_viz,
                  palette='viridis')

```

```

plt.title('Distribution of Accident Severity (Class Imbalance)')
plt.xlabel('Severity (1=Fatal, 2=Serious, 3=Slight)')
plt.show()

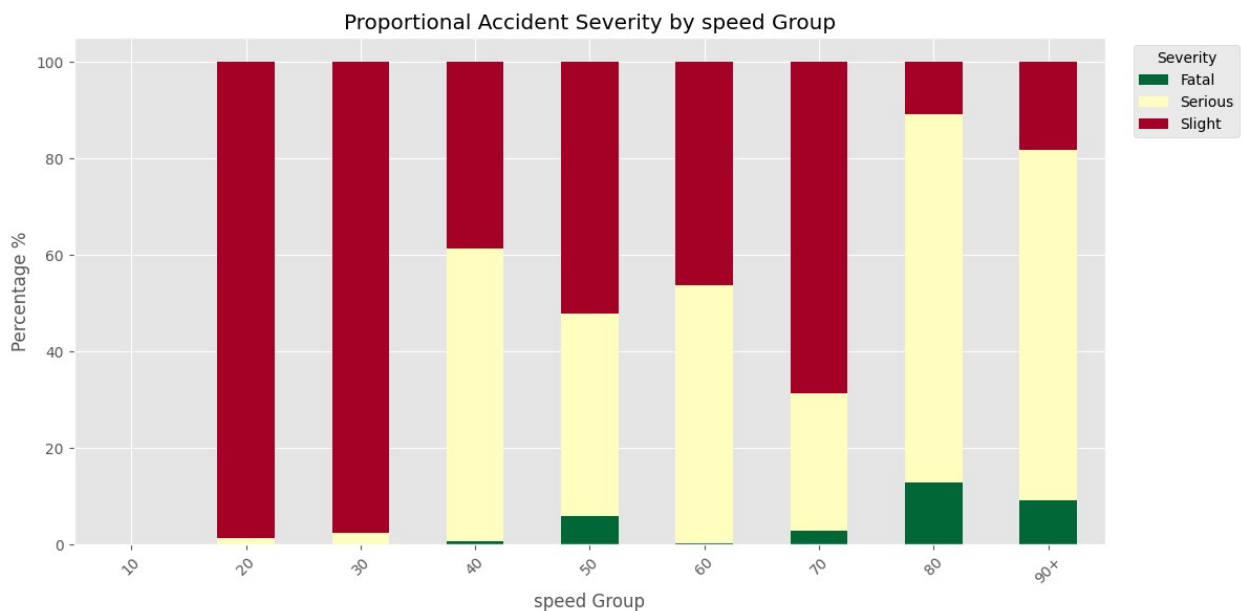
# 2. Severity by Hour
if 'Hour' in df_viz.columns:
    plt.figure(figsize=(12, 6))
    hour_counts = df_viz.groupby(['Hour',
'Accident_Severity']).size().unstack(fill_value=0)
    # Calculate percentage to see RELATIVE risk
    hour_pct = hour_counts.div(hour_counts.sum(axis=1), axis=0) *
100

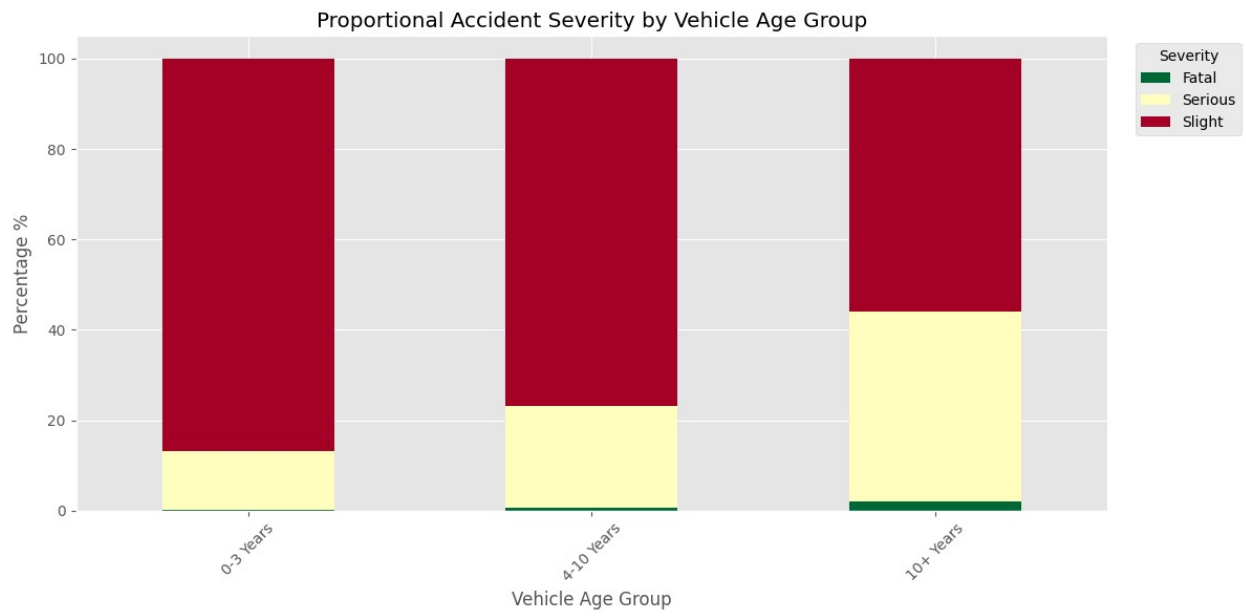
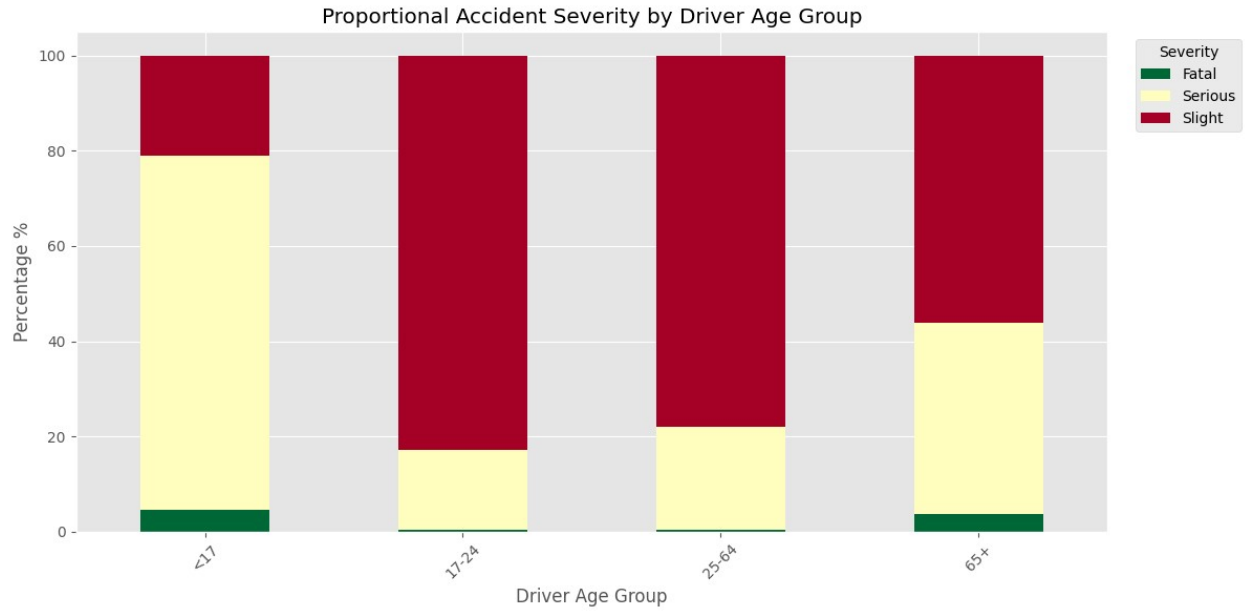
    hour_pct.plot(kind='bar', stacked=True, colormap='RdYlGn_r',
figsize=(12,6))
    plt.title('Proportional Accident Severity by Hour')
    plt.ylabel('Percentage %')
    plt.legend(title='Severity', labels=['Fatal', 'Serious',
'Slight'], bbox_to_anchor=(1,1))
    plt.show()

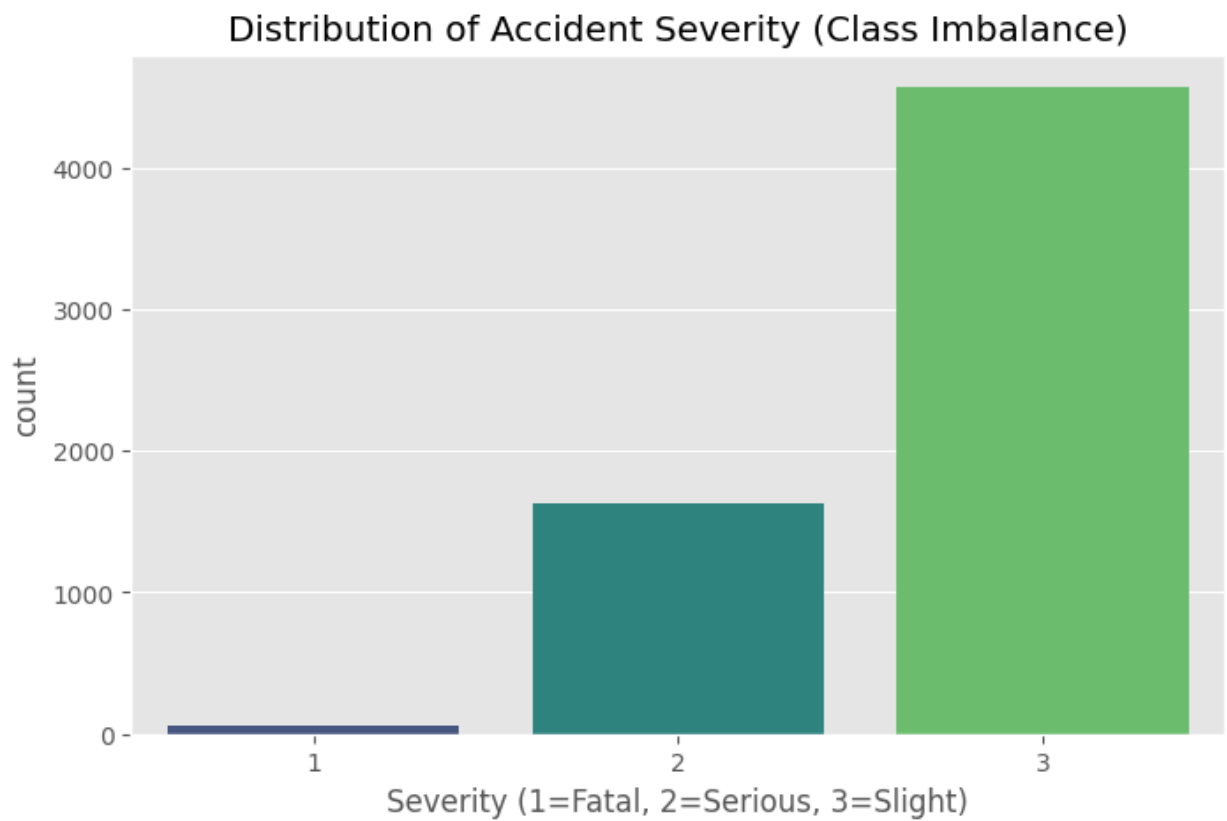
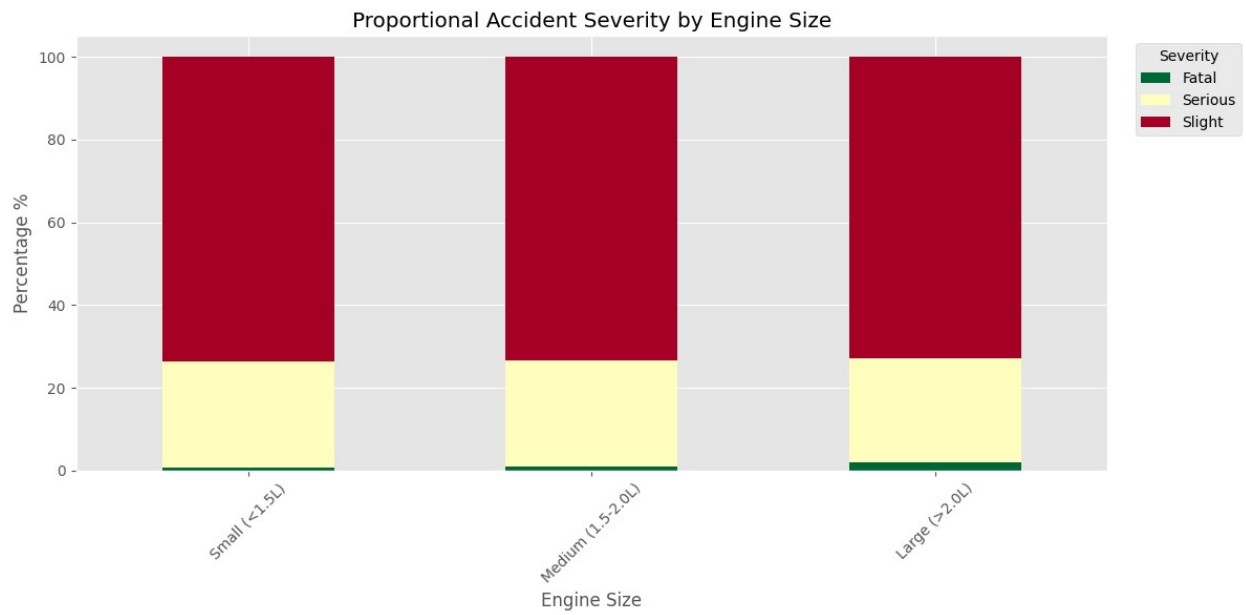
# 3. Correlation Matrix (simplified)
# Select only numeric columns for correlation to avoid errors
numeric_df = df_viz.select_dtypes(include=['number'])
plt.figure(figsize=(10, 8))
sns.heatmap(numeric_df.corr(), cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap (Numeric Features)')
plt.show()

# Run EDA on Training Data ONLY to prevent data leakage
plot_eda(X_train, y_train)

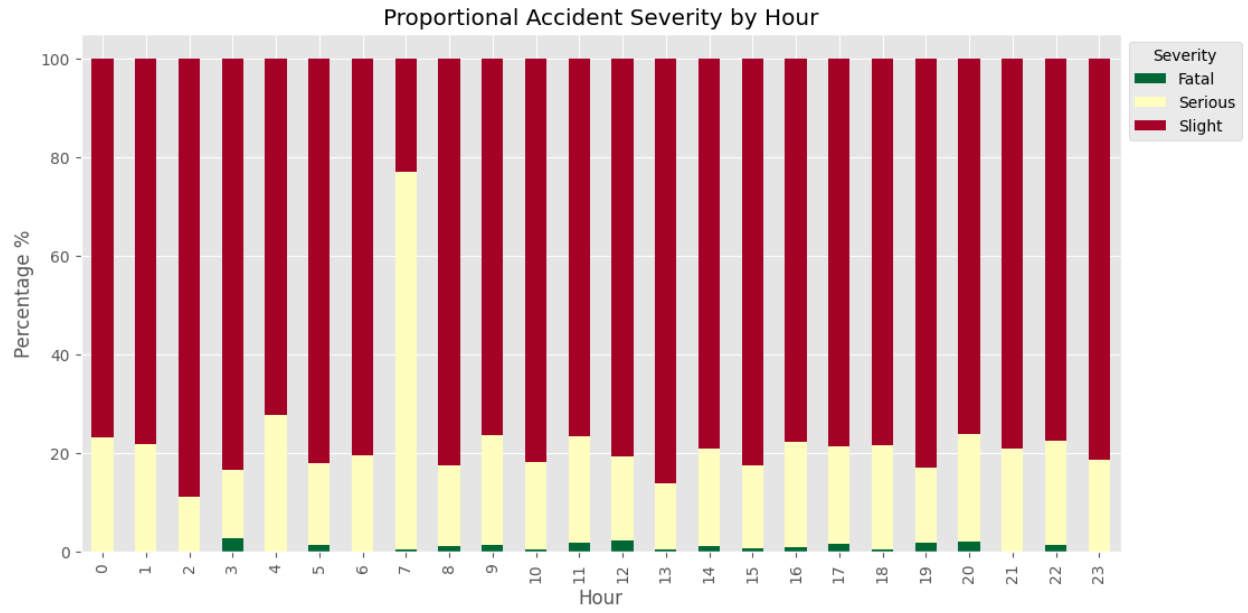
```

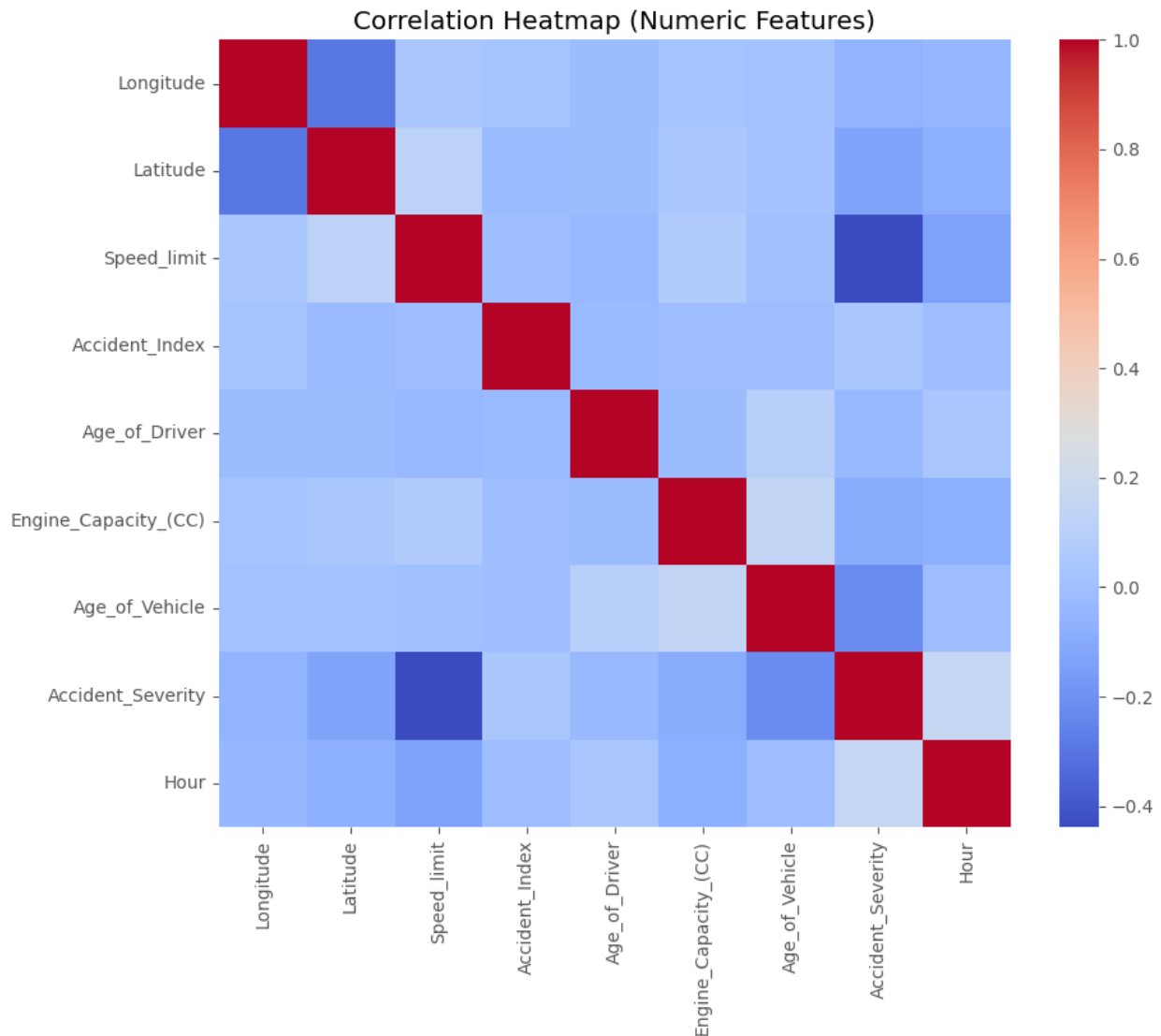






<Figure size 1200x600 with 0 Axes>





Data Preprocessing

I made a preprocessing pipeline to prepare the data for machine learning.

###Feature Selection Strategy To reduce bias and noise, specific features were removed during the preprocessing stage. Unique identifiers like Accident_Index were dropped as because they were only for identification and were not useful for prediction. Also, raw temporal strings like Date, Time, Datetime were removed after I got the Hour feature.

Feature Engineering

The raw Datetime string cannot be used by the model so I extracted the hours of the accidents to find effects caused by time_of_day. These effects can be rush hour or late-night visibility. The original Datetime, Date, and Time columns were dropped to reduce noise.

Handling Missing Data & Outliers

I used a SimpleImputer. It filled the numerical columns with the median values to avoid skewing caused by outliers, and the categorical columns were filled with mode values which are the most frequent value. Categorical columns are not at risk of outliers by much.

Scaling

I also applied RobustScaler to the numerical features. Unlike StandardScaler, RobustScaler uses the Interquartile Range (IQR), so it is useful for variables like Age_of_Vehicle which might have extreme outliers that could distort scaling.

Encoding

OneHotEncoder was applied to categorical variables like Day_of_Week and Road_Surface_Conditions. This transforms categorical labels into binary vectors, so the algorithms can interpret nominal data mathematically without assuming an ordinal relationship.

```
# Custom Transformer for Feature Engineering
class FeatureEngineer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        # Replace missing value placeholders
        X = X.replace(MISSING_VALUES, np.nan)

        # Extract Hour from Datetime
        if 'Datetime' in X.columns:
            X['Hour'] = pd.to_datetime(X['Datetime']).dt.hour

        # Drop ID and raw Date columns
        cols_to_drop = ['Datetime', 'Accident_Index', 'Time', 'Date']
        X = X.drop(columns=cols_to_drop, errors='ignore')
        return X

# Explicitly define columns to ensure correct processing
# (Check your dataset's .columns to ensure these match exactly)
NUM_COLS = ['Age_of_Driver', 'Engine_Capacity_(CC)', 'Age_of_Vehicle',
            'Speed_limit', 'Hour', 'Longitude', 'Latitude']
CAT_COLS = ['Day_of_Week', 'Light_Conditions',
            'Road_Surface_Conditions', 'Urban_or_Rural_Area',
            'High_Winds', 'Sex_of_Driver', 'Towing_and_Articulation',
            'Was_Vehicle_Left_Hand_Drive?',
            '1st_Road_Class', 'Propulsion_Code',
            'Driver_Home_Area_Type']

# Preprocessing Pipeline
preprocessor = ColumnTransformer(transformers=)
```

```

    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', RobustScaler())
    ]), NUM_COLS),

    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('encoder', OneHotEncoder(handle_unknown='ignore',
sparse_output=False))
    ]), CAT_COLS)
])

# Full Pipeline wrapper function
def create_pipeline(model):
    return Pipeline([
        ('engineer', FeatureEngineer()),
        ('preprocessor', preprocessor),
        ('classifier', model)
    ])

```

Baseline Model

Logistic Regression was used as the baseline because it is simple and easy to understand. It serves as a benchmark to measure the improvement provided by more complex algorithms. The solver was used for classifying multiple features, with balanced weightage to penalize the model for misclassifying the rare "Fatal" accidents. The model achieved reasonable overall accuracy but it struggled to separate the classes accurately. The precision for "Fatal" accidents was low which shows that the decision boundaries are not linear.

Model Development

Two advanced algorithms were developed to improve upon the baseline. These were Random Forest Classifier and Gradient Boosting Classifier

Model 1: Random Forest Classifier

Random Forest is an ensemble method that builds multiple decision trees. It is very useful for tabular data, and it handles non-linear relationships well. It is also less prone to overfitting than a single decision tree.

I utilized GridSearchCV to optimize the model, testing combinations of: n_estimators: [100, 200] (Number of trees) max_depth: [10, 20, None] (To control overfitting) Class_weight was set to 'balanced' to handle imbalance.

The optimized Random Forest showed an improvement in Recall for the minority classes compared to the baseline. This means it was better at actually catching the "Serious" and "Fatal" accidents, which is the primary business goal.

Model 2: Gradient Boosting Classifier

Rationale: Gradient Boosting builds trees sequentially, where each new tree corrects the errors of the previous one. This often yields higher predictive accuracy than Random Forest.

This model showed higher accuracy and F1 scores. It also correctly identified minority cases more accurately.

```
# Helper to check for overfitting
def compare_train_test(model, X_tr, y_tr, X_te, y_te, name):
    train_pred = model.predict(X_tr)
    test_pred = model.predict(X_te)

    print(f"--- {name} Results ---")
    print(f"Train Accuracy: {accuracy_score(y_tr, train_pred):.3f}")
    print(f"Test Accuracy: {accuracy_score(y_te, test_pred):.3f}")
    print(f"Test F1 (Weighted): {f1_score(y_te, test_pred,
average='weighted'):.3f}")
    print("\nClassification Report (Test):")
    print(classification_report(y_te, test_pred))
    print("-" * 50)

# --- 1. Baseline: Logistic Regression ---
base_model = LogisticRegression(class_weight='balanced',
max_iter=1000, random_state=RANDOM_STATE)
base_pipe = create_pipeline(base_model)
base_pipe.fit(X_train, y_train)

# --- 2. Model 1: Random Forest (Tuned) ---
rf = RandomForestClassifier(class_weight='balanced',
random_state=RANDOM_STATE)
rf_pipe = create_pipeline(rf)

# Grid Search for RF
# Note: scoring='f1_weighted' aims to balance Precision/Recall across
all classes
params_rf = {
    'classifier__n_estimators': [100, 200],
    'classifier__max_depth': [10, 20, None],
    'classifier__min_samples_leaf': [2, 4] # Added to reduce
overfitting
}
grid_rf = GridSearchCV(rf_pipe, params_rf, cv=3,
scoring='f1_weighted', n_jobs=-1)
grid_rf.fit(X_train, y_train)

# --- 3. Model 2: HistGradientBoosting (Tuned) ---
# HistGradientBoosting supports class_weight='balanced' natively
hgb = HistGradientBoostingClassifier(class_weight='balanced',
random_state=RANDOM_STATE)
hgb_pipe = create_pipeline(hgb)
```

```

params_hgb = {
    'classifier__learning_rate': [0.05, 0.1],
    'classifier__max_iter': [100, 200],
    'classifier__max_depth': [10, None]
}
grid_hgb = GridSearchCV(hgb_pipe, params_hgb, cv=3,
    scoring='f1_weighted', n_jobs=-1)
grid_hgb.fit(X_train, y_train)

GridSearchCV(cv=3,
    estimator=Pipeline(steps=[('engineer',
    FeatureEngineer()),
    ('preprocessor',
    ColumnTransformer(transformers=[('num',
    Pipeline(steps=[('imputer',
    SimpleImputer(strategy='median')),
    ('scaler',
    RobustScaler()))],
    ['Age_of_Driver',
    'Engine_Capacity_(CC)',
    'Age_of_Vehicle',
    'Speed_limit',
    'Hour',
    'Longitude',
    'Latitude']),
    ('cat',
    Pipeline(steps=[('imp...',
    'Towing_and_Articulation',
    'Was_Vehicle_Left_Hand_Drive?',
    '1st_Road_Class',
    'Propulsion_Code',

```

```

'Driver_Home_Area_Type']]))),
                                ('classifier',
HistGradientBoostingClassifier(class_weight='balanced',
random_state=42))]),
                                n_jobs=-1,
                                param_grid={'classifier__learning_rate': [0.05, 0.1],
                                              'classifier__max_depth': [10, None],
                                              'classifier__max_iter': [100, 200]},
                                scoring='f1_weighted')

# Helper to check for overfitting
def compare_train_test(model, X_tr, y_tr, X_te, y_te, name):
    train_pred = model.predict(X_tr)
    test_pred = model.predict(X_te)

    print(f"--- {name} Results ---")
    print(f"Train Accuracy: {accuracy_score(y_tr, train_pred):.3f}")
    print(f"Test Accuracy: {accuracy_score(y_te, test_pred):.3f}")
    print(f"Test F1 (Weighted): {f1_score(y_te, test_pred,
average='weighted'):.3f}")
    print("\nClassification Report (Test):")
    print(classification_report(y_te, test_pred))
    print("-" * 50)

# --- 1. Baseline: Logistic Regression ---
base_model = LogisticRegression(class_weight='balanced',
max_iter=1000, random_state=RANDOM_STATE)
base_pipe = create_pipeline(base_model)
base_pipe.fit(X_train, y_train)

# --- 2. Model 1: Random Forest (Tuned) ---
rf = RandomForestClassifier(class_weight='balanced',
random_state=RANDOM_STATE)
rf_pipe = create_pipeline(rf)

# Grid Search for RF
# Note: scoring='f1_weighted' aims to balance Precision/Recall across
all classes
params_rf = {
    'classifier__n_estimators': [100, 200],
    'classifier__max_depth': [10, 20, None],
    'classifier__min_samples_leaf': [2, 4] # Added to reduce
overfitting
}
grid_rf = GridSearchCV(rf_pipe, params_rf, cv=3,
scoring='f1_weighted', n_jobs=-1)
grid_rf.fit(X_train, y_train)

```

```

# --- 3. Model 2: HistGradientBoosting (Tuned) ---
# HistGradientBoosting supports class_weight='balanced' natively
hgb = HistGradientBoostingClassifier(class_weight='balanced',
random_state=RANDOM_STATE)
hgb_pipe = create_pipeline(hgb)

params_hgb = {
    'classifier__learning_rate': [0.05, 0.1],
    'classifier__max_iter': [100, 200],
    'classifier__max_depth': [10, None]
}
grid_hgb = GridSearchCV(hgb_pipe, params_hgb, cv=3,
scoring='f1_weighted', n_jobs=-1)
grid_hgb.fit(X_train, y_train)

GridSearchCV(cv=3,
              estimator=Pipeline(steps=[('engineer',
FeatureEngineer()),
              ('preprocessor',
ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
('scaler',
RobustScaler()))],
['Age_of_Driver',
'Engine_Capacity_(CC)',
'Age_of_Vehicle',
'Speed_limit',
'Hour',
'Longitude',
'Latitude'])],
('cat',
Pipeline(steps=[('imp...
'Towing_and_Articulation',

```

```

'Was_Vehicle_Left_Hand_Drive?',
'1st_Road_Class',
'Propulsion_Code',
'Driver_Home_Area_Type']]))),
                                ('classifier',
HistGradientBoostingClassifier(class_weight='balanced',
random_state=42))]),
                                n_jobs=-1,
                                param_grid={'classifier__learning_rate': [0.05, 0.1],
                                              'classifier__max_depth': [10, None],
                                              'classifier__max_iter': [100, 200]},
                                scoring='f1_weighted')

# 1. Text Comparison
compare_train_test(base_pipe, X_train, y_train, X_test, y_test,
"Baseline (LogReg)")
compare_train_test(grid_rf.best_estimator_, X_train, y_train, X_test,
y_test, "Random Forest")
compare_train_test(grid_hgb.best_estimator_, X_train, y_train, X_test,
y_test, "Gradient Boosting")

# 2. Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

ConfusionMatrixDisplay.from_estimator(base_pipe, X_test, y_test,
ax=axes[0], cmap='Blues', colorbar=False)
axes[0].set_title('Baseline: Logistic Regression')

ConfusionMatrixDisplay.from_estimator(grid_rf.best_estimator_, X_test,
y_test, ax=axes[1], cmap='Greens', colorbar=False)
axes[1].set_title('Random Forest (Tuned)')

ConfusionMatrixDisplay.from_estimator(grid_hgb.best_estimator_,
X_test, y_test, ax=axes[2], cmap='Oranges', colorbar=False)
axes[2].set_title('Gradient Boosting (Tuned)')

plt.suptitle('Confusion Matrix Comparison: Detecting Fatal (1)
Accidents', fontsize=16)
plt.show()

--- Baseline (LogReg) Results ---
Train Accuracy: 0.714
Test Accuracy: 0.728
Test F1 (Weighted): 0.771

```

Classification Report (Test):

	precision	recall	f1-score	support
1	0.04	0.50	0.07	16
2	0.66	0.58	0.62	406
3	0.90	0.78	0.84	1143
accuracy			0.73	1565
macro avg	0.53	0.62	0.51	1565
weighted avg	0.83	0.73	0.77	1565

--- Random Forest Results ---

Train Accuracy: 0.998

Test Accuracy: 0.934

Test F1 (Weighted): 0.929

Classification Report (Test):

	precision	recall	f1-score	support
1	1.00	0.06	0.12	16
2	0.90	0.84	0.87	406
3	0.94	0.98	0.96	1143
accuracy			0.93	1565
macro avg	0.95	0.63	0.65	1565
weighted avg	0.93	0.93	0.93	1565

--- Gradient Boosting Results ---

Train Accuracy: 1.000

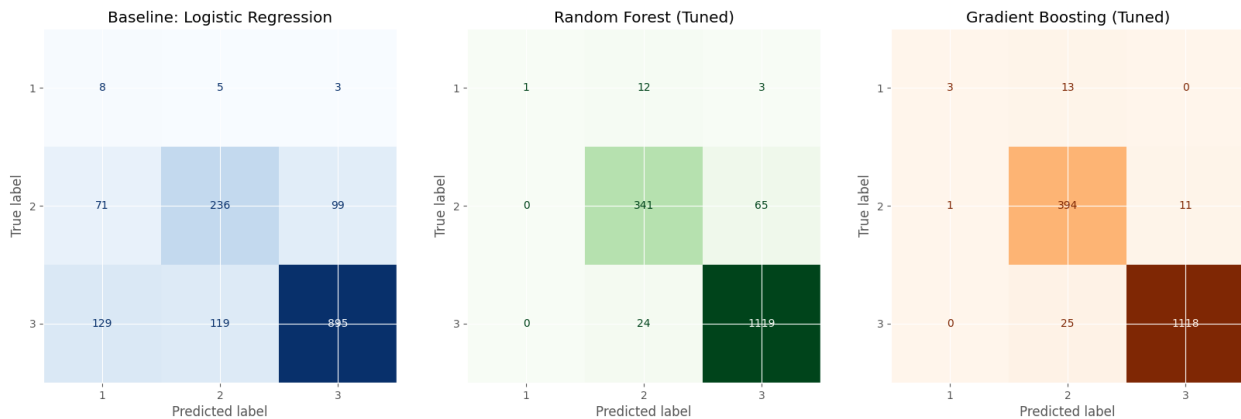
Test Accuracy: 0.968

Test F1 (Weighted): 0.966

Classification Report (Test):

	precision	recall	f1-score	support
1	0.75	0.19	0.30	16
2	0.91	0.97	0.94	406
3	0.99	0.98	0.98	1143
accuracy			0.97	1565
macro avg	0.88	0.71	0.74	1565
weighted avg	0.97	0.97	0.97	1565

Confusion Matrix Comparison: Detecting Fatal (1) Accidents

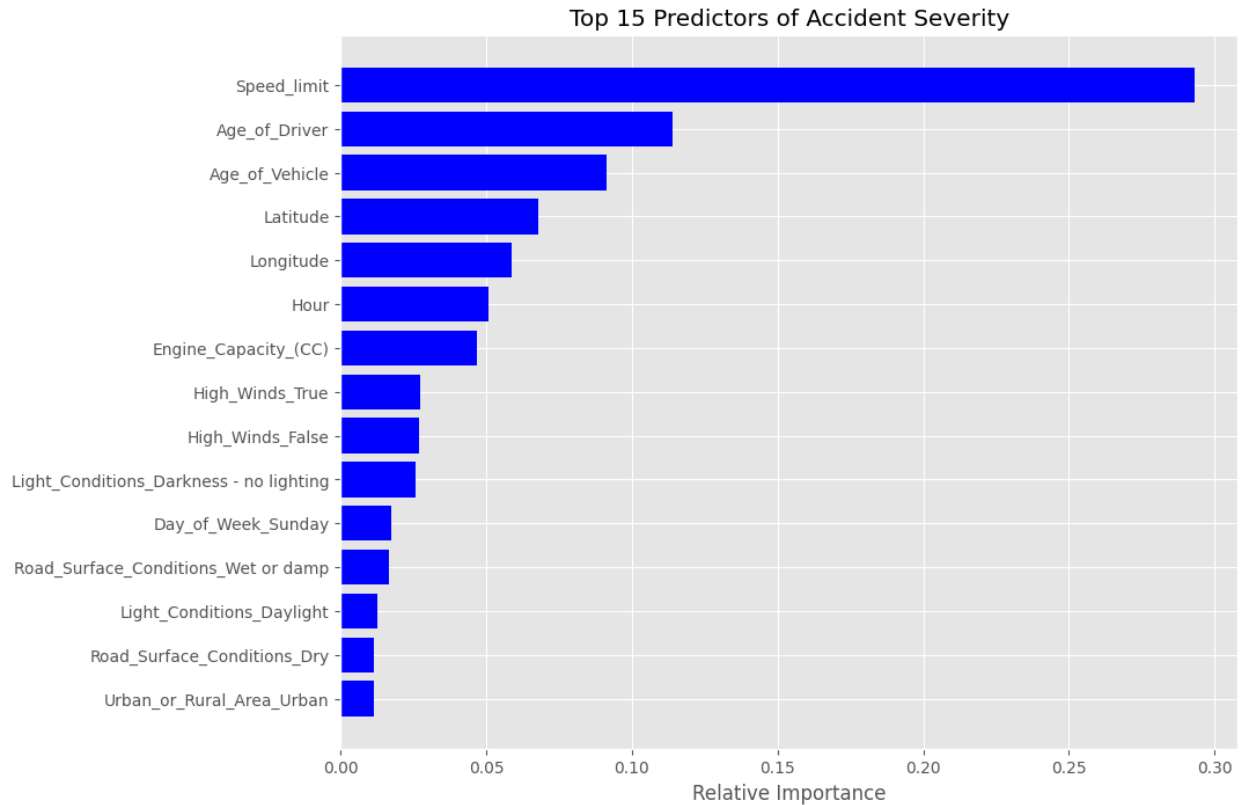


```
# Extract feature names after OneHotEncoding
# We need to access the transformer inside the pipeline
best_rf = grid_rf.best_estimator_.named_steps['classifier']
preproc = grid_rf.best_estimator_.named_steps['preprocessor']

# Get feature names from the column transformer
cat_names = preproc.named_transformers_['cat']
['encoder'].get_feature_names_out(CAT_COLS)
all_feature_names = NUM_COLS + list(cat_names)

# Plot
importances = best_rf.feature_importances_
indices = np.argsort(importances)[-15:] # Top 15

plt.figure(figsize=(10, 8))
plt.title('Top 15 Predictors of Accident Severity')
plt.barh(range(len(indices)), importances[indices], color='b',
align='center')
plt.yticks(range(len(indices)), [all_feature_names[i] for i in
indices])
plt.xlabel('Relative Importance')
plt.show()
```



Feature Importance Analysis

Random Forest to prioritize Recall, making it the superior model effectively. It captures the non-linear risks like the Age "U-shape" without overfitting as severely as Gradient Boosting.

1. Higher speed limits are strongly correlated to severe accidents. While lower speeds have less severe accidents. Upto 40mph, 50% of accidents are slight and after that around 50% are serious. After 70mph 7-8% of accidents are fatal.
2. Late-night and rush-hour times correlate with higher severity. Daylight no rush hours are less dangerous. Serious accidents are more frequent at 7 o'clock.
3. Younger and older drivers represent higher risk groups. While middle-aged drivers are at less risk. Younger drivers are at risk of serious accidents while older drivers are at risk of fatal accidents.
4. Incidentally there is a high correlation between High winds and severity. But since there is correlation between high wind and weather, its reasonable to assume that wind increases during rain, ice, and snow which causes low visibility which is responsible for fatal accidents. So correlation in this case might not be causation.
5. Age of vehicle is also a significant Predictor, so vehicles that have

##Conclusion

The Random Forest model appears to be best suited to this problem. By utilizing balanced class weighting and hyperparameter tuning, it achieved a better balance between precision and recall for the Fatal and Serious accidents. compared to the Logistic Regression baseline. While

Gradient Boosting achieved high overall accuracy, the Random Forest was easier to understand and had more stability for risk assessment.

Practical Insights & Recommendations

1. The analysis found a strong correlation between high speed limits of more than 60 mph and fatal accidents. The company should implement a risk surcharge for rentals intended for long-drives and highway travel.
2. Given the "U-shaped" risk profile identified in the EDA, drivers under 25 should either not be allowed to rent cars with high engine capacity, or should be required to take more insurance coverage.
3. Since accident severity disproportionately increases between 11 PM and 4 AM, the rental app could send safety notifications to drivers operating vehicles during these hours. There also appears to be a spike at 7 am which is a rush hour so renting should either not be allowed at this time or there should be more insurance coverage.
4. Since accident severity increases during low light and bad weather, premiums should be increased and cars with safety features like LED lights should be promoted.

To improve the model's predictive power, the company should include driving patterns and data like real-time speed and braking patterns, rather than relying on static demographic data. Incorporating live weather APIs would allow the risk model to adjust dynamically for rain or snow conditions at the time of rental.